

DOCUMENTO DE ARQUITETURA E DESENVOLVIMENTO DO PROJETO

Plataforma Serverless Enterprise de Ingestão, Notificações e Observabilidade na AWS

Projeto: Plataforma de Gestão de Feedbacks & Notificação em Tempo Real
Equipe Responsável: Arquitetura Cloud & Engenharia de Software
Tecnologias Core: AWS Serverless (API Gateway, Lambda Python 3.12, DynamoDB Streams, SQS + DLQ, SNS, SES, CloudWatch), Terraform (IaC)
Repositório GitHub: feedback-management (CI/CD Automatizado via GitHub Actions)
Status: Infraestrutura e Código Serverless Concluídos | Observabilidade e Segurança Ativadas
Data da Documentação: Julho / 2026

1. Fase de Idealização e Arquitetura da Solução

A fase de idealização estabeleceu um modelo arquitetural baseado em **microsserviços Serverless e Orientados a Eventos (EDA)** na AWS, eliminando custos fixos de infraestrutura e garantindo escalabilidade automática. Abaixo apresenta-se o diagrama arquitetural da solução, detalhando o fluxo de ingestão REST, persistência NoSQL, triagem automatizada de urgência, desacoplamento assíncrono e a integração da **Dead Letter Queue (DLQ)** para resiliência operacional.

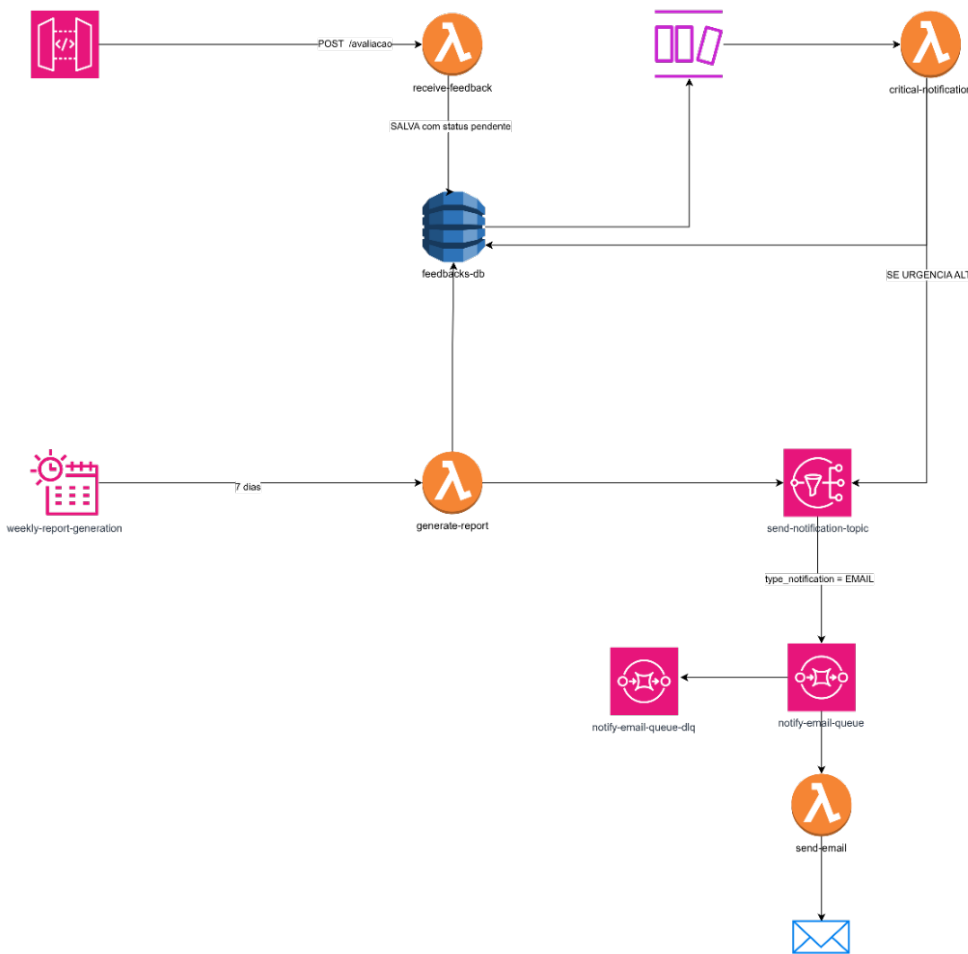


Figura 1: Diagrama de Arquitetura da Solução Serverless com Integração de DLQ

Detalhamento do Fluxo de Dados e Resiliência com DLQ:

- 1. Ingestão REST (API Gateway & Lambda receive-feedback):** O endpoint POST /avaliacao recebe requisições REST HTTP. O API Gateway realiza a validação de JSON Schema e verifica a chave de API (x-api-key). A Lambda *receive-feedback* gera um UUID v4 único e persiste a avaliação no Amazon DynamoDB (feedbacks-db) com status inicial pendente.

- **2. Triagem Reativa (DynamoDB Streams & Lambda critical-notification / evaluate-urgency):** A inserção no DynamoDB aciona o DynamoDB Stream em tempo real, que dispara a Lambda de avaliação de urgência. A função lê as regras operacionais no SSM Parameter Store (/config/urgency_thresholds). Se a nota for ≤ 4 , o registro é atualizado para urgência ALTA e um alerta é emitido.
- **3. Roteamento Assíncrono (Amazon SNS & Amazon SQS):** Notificações de urgência ALTA são publicadas no tópico Amazon SNS (send-notification-topic) e direcionadas para a fila Amazon SQS (notify-email-queue), garantindo desacoplamento e absorção de picos de carga.
- **4. Tolerância a Falhas e Resiliência (SQS Dead Letter Queue - DLQ):** Para prevenir descarte de mensagens em caso de indisponibilidades temporárias (ex: limites de quota no SES ou oscilações de rede), a fila principal possui uma **Dead Letter Queue (DLQ - notify-email-queue-dlq)** acoplada via Redrive Policy. Mensagens que falharem após 3 tentativas (maxReceiveCount = 3) são isoladas na DLQ para auditoria e reprocessamento posterior.
- **5. Disparo de E-mails HTML (Lambda send-email & Amazon SES):** A Lambda *send-email* consome a fila SQS via Event Source Mapping e aciona o Amazon SES, que renderiza e envia e-mails corporativos estilizados usando o template HTML CriticalAlertTemplate.
- **6. Relatórios Semanal e Sob Demanda (EventBridge Scheduler & Lambda generate-report):** A cada 7 dias, o Amazon EventBridge Scheduler dispara a Lambda *generate-report*, que executa consultas agregadas na GSI DateIndex do DynamoDB e emite relatórios estatísticos consolidados via SES (WeeklyReportTemplate). Adicionalmente, administradores podem solicitar relatórios sob demanda via POST /relatorio.

2. Fase de Levantamento de Requisitos

A especificação do sistema baseou-se na divisão clara entre requisitos funcionais e não-funcionais, estabelecendo os critérios de aceite técnicos e operacionais para a plataforma:

Categoria	ID	Descrição do Requisito	Componente AWS Responsável
Funcional	RF-01	Ingestão de avaliações via API REST com validação de payload JSON.	API Gateway / Lambda receive-feedback
Funcional	RF-02	Triagem automática de urgência em tempo real baseada na nota da avaliação.	DynamoDB Stream / Lambda evaluate-urgency
Funcional	RF-03	Notificação imediata por e-mail para a gestão ao identificar notas críticas (nota ≤ 4).	SNS / SQS / Lambda send-email / SES
Funcional	RF-04	Geração automatizada de relatórios estatísticos semanais e sob demanda.	EventBridge / Lambda generate-report
Funcional	RF-05	Envio de e-mails formatados em templates HTML corporativos.	Amazon SES Templates
Não-Funcional	RNF-01	Arquitetura 100% Serverless com custo sob demanda e zero gestão de servidores.	AWS Lambda / DynamoDB / SQS
Não-Funcional	RNF-02	Resiliência e tolerância a falhas com isolamento de falhas via SQS DLQ.	SQS Dead Letter Queue (DLQ)
Não-Funcional	RNF-03	Controle de Acesso Granular (RBAC) e rate-limiting por perfil de usuário via API Keys.	API Gateway Usage Plans
Não-Funcional	RNF-04	Automação integral da infraestrutura como código (IaC) e estado remoto seguro.	Terraform / S3 Remote State
Não-Funcional	RNF-05	Observabilidade nativa com dashboards visuais de métricas de negócio e logs.	Amazon CloudWatch Dashboards & Logs

3. Fase de Gestão de Projetos e Atividades (Quadro Trello)

A condução do desenvolvimento seguiu a metodologia ágil Kanban através do quadro Trello, garantindo visibilidade e rastreabilidade sobre as entregas:

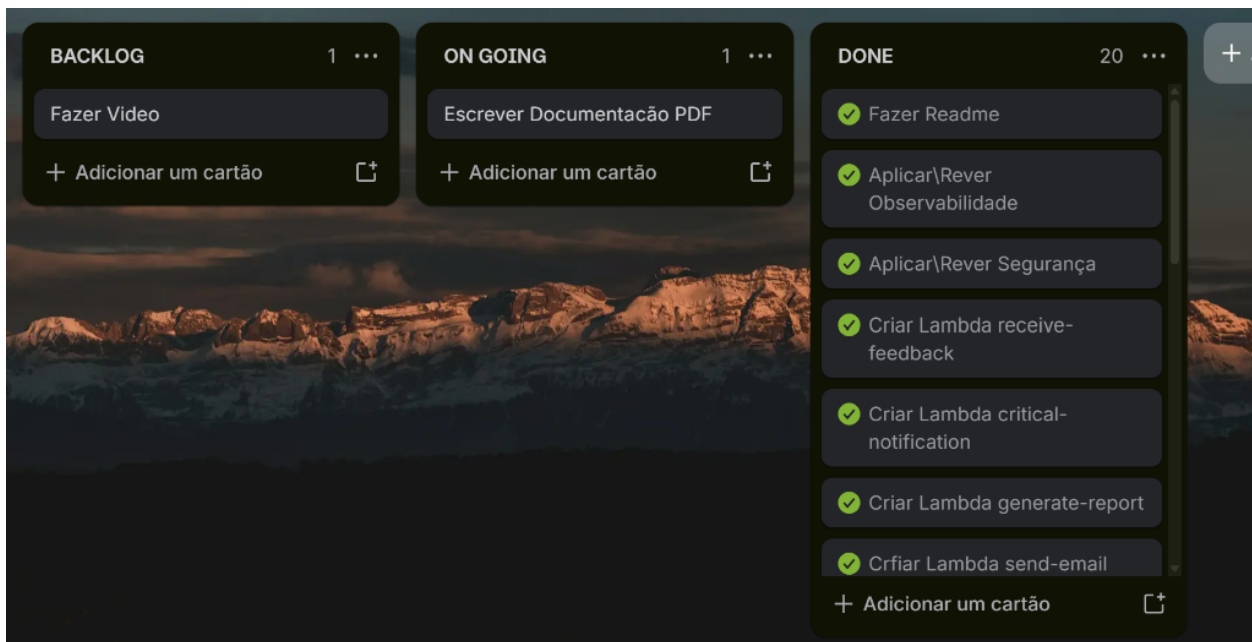


Figura 2: Acompanhamento de Atividades e Ciclo de Vida do Projeto no Quadro Trello

4. Fase de Desenvolvimento do Core da Aplicação e IaC (Terraform)

Toda a infraestrutura do projeto foi codificada em HCL utilizando o **Terraform (v1.5+)** com estado remoto em S3. As funções Serverless foram modularizadas com base na regra de responsabilidade única (SRP):

Arquivo Terraform	Recursos AWS Provisionados	Responsabilidade na Arquitetura
apigateway.tf	REST API Gateway, JSON Schema Models, Request Validators, Usage Plans, API Keys	Exposição e proteção dos endpoints REST com rate limiting e autorização por perfil.
dynamodb.tf	Tabela DynamoDB feedbacks_db, DynamoDB Stream, GSI DateIndex	Armazenamento NoSQL com streaming reativo e indexação para consultas por data.
sqs.tf / sns.tf	SQS notify-email-queue, SQS DLQ notify-email-queue-dlq, SNS Topic	Fila assíncrona principal e Dead Letter Queue com Redrive Policy (maxReceiveCount=3).
lambdas.tf	4x AWS Lambda (Python 3.12), Event Source Mappings, Environment Variables	Empacotamento automático zip e deploy dos executáveis Serverless desacoplados.
iam.tf	IAM Roles dedicadas e IAM Inline Policies refinadas	Aplicação rigorosa do Princípio do Menor Privilégio para cada serviço Lambda.
ssm.tf	SSM Parameter Store (/config/urgency_thresholds)	Gerenciamento centralizado de parâmetros dinâmicos sem necessidade de re-deploy.
ses_templates.tf	Amazon SES Email Templates HTML (CriticalAlertTemplate, WeeklyReportTemplate)	Renderização de e-mails corporativos estilizados com placeholders dinâmicos.
cloudwatch.tf	CloudWatch Dashboard, Metric Filters, CloudWatch Alarms	Coleta de logs estruturados e exibição gráfica de métricas de negócio e operacionais.
eventbridge.tf	Amazon EventBridge Scheduler Rule (Cron de 7 dias)	Gatilho temporal agendado para emissão automatizada de relatórios consolidados.

Evidências de Notificações por E-mail (Amazon SES):

Abaixo apresentam-se os prints reais das mensagens recebidas como resultado das invocações do sistema: (1) Alerta de Avaliação Crítica instantâneo e (2) Relatório Semanal de Avaliações consolidado.

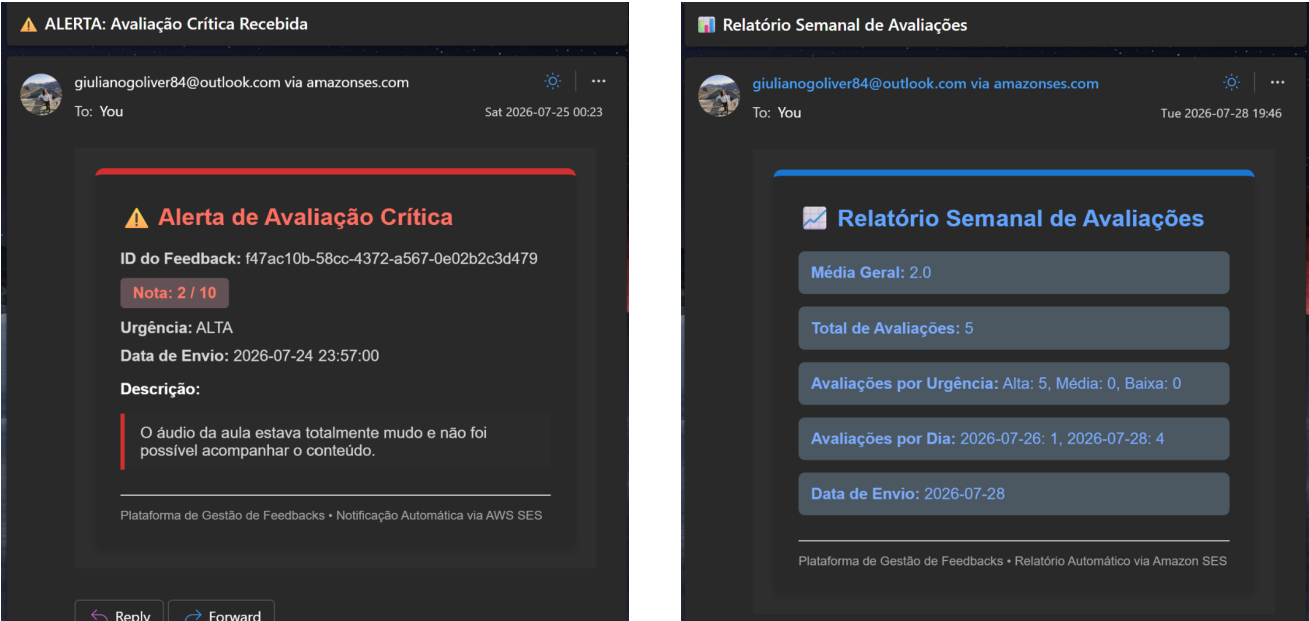


Figura 3: Evidências de E-mails Recebidos via Amazon SES - Alerta de Avaliação Crítica (esquerda) e Relatório Semanal de Avaliações (direita)

5. Fase de Segurança, Qualidade e Testes

Controle de Acesso Granular (RBAC & Throttling):

A segurança da API foi implementada através de **API Keys** associadas a **Usage Plans** do API Gateway, criando um modelo de Controle de Acesso Baseado em Funções (RBAC) com limites diferenciados:

Perfil do Solicitante	Header Obrigatório	Endpoints Permitidos	Throttling & Quotas de Acesso
Aluno / Cliente	x-api-key: STUDENT_KEY	POST /avaliacao	Rate Limit: 10 req/s Burst: 20 Quota: 10.000 requisições/mês
Administrador / Gestão	x-api-key: ADMIN_KEY	POST /avaliacao POST /relatorio	Rate Limit: 20 req/s Burst: 50 Quota: 50.000 requisições/mês

■ **Proteção Automática no Edge:** Caso um usuário com API Key do perfil *Aluno* tente invocar o endpoint restrito `POST /relatorio`, o API Gateway rejeita a chamada com **HTTP 403 Forbidden** antes de atingir qualquer função Lambda.

Automação de CI/CD (GitHub Actions):

- **01-pr-pipeline.yml:** Validação sintática e de formatação (terraform fmt, validate, plan) em Pull Requests para a branch main.
- **02-deploy-on-merge.yml:** Deploy automatizado de infraestrutura via terraform apply no merge.

Testes de Integração (Postman Collection):

Disponibilizada no repositório em `collection/Feedback.postman_collection.json` para testes de integração de ponta a ponta.

6. Fase de Observabilidade e Monitoramento (Amazon CloudWatch)

A observabilidade foi estruturada nativamente com o **Amazon CloudWatch**, disponibilizando o dashboard ``FeedbackPlatform-Business-Dashboard`` com visão em tempo real de métricas e logs:

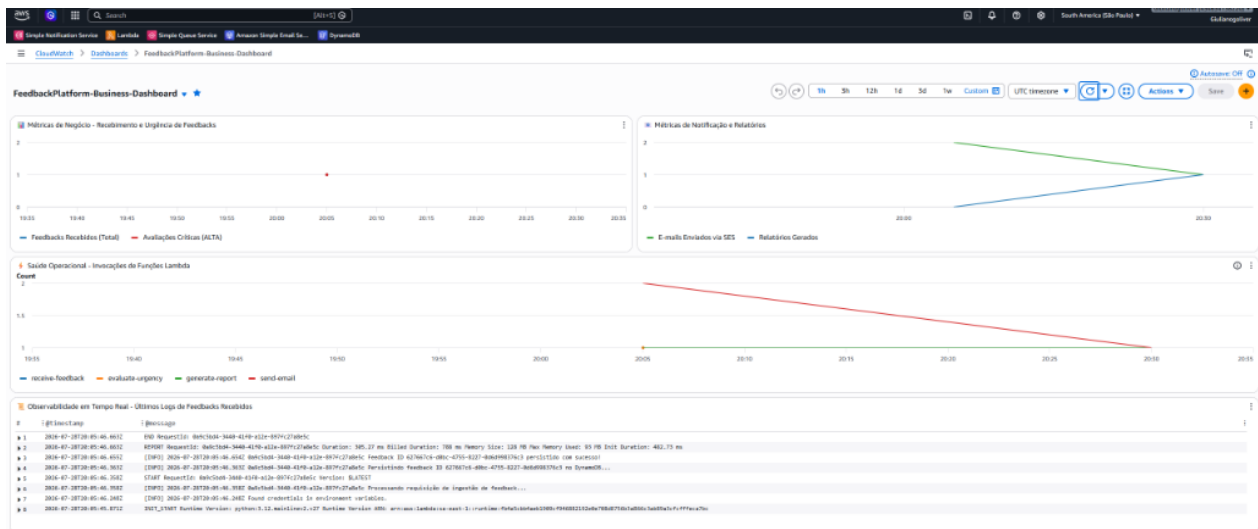


Figura 4: Dashboard Executivo de Observabilidade, Métricas de Negócio e Logs no Amazon CloudWatch

- **Métricas de Negócio (Recebimento e Urgência):** Gráficos acompanhando a taxa total de feedbacks capturados e a identificação de avaliações críticas (urgência ALTA).
- **Métricas de Notificação e Relatórios:** Monitoramento de e-mails enviados via Amazon SES e relatórios consolidados emitidos.
- **Saúde Operacional das Lambdas:** Contagem de invocações, duração de execução (latência) e erros das 4 funções Serverless.
- **Observabilidade em Tempo Real (CloudWatch Logs Insights):** Widget de consulta aos últimos logs de execuções com RequestId, memória utilizada e persistência.